

Week 1

Team project update/progress

Table of Contents

- 01 Summary of Weekly Progress
- 02 Development Tools (Github)
- 03 Getting Familiar with the code
- 04 Design and Prototype Development

Weekly Progress Summary

GitHub	Team familiarized with basic concepts
Documentation	Reviewed and analyzed Code/PDF documentation
Architecture	Studied agent roles and system architecture
Onboarding	Team discussed about the UI design scope and objectives
UI Design	Explored design options and created prototypes
Selection	Compared prototypes and selected final design

GitHub Familiarization

Repository

Centralized file and code management

Version Control

Change tracking and version history

Task Division

Issue-based work assignment and tracking

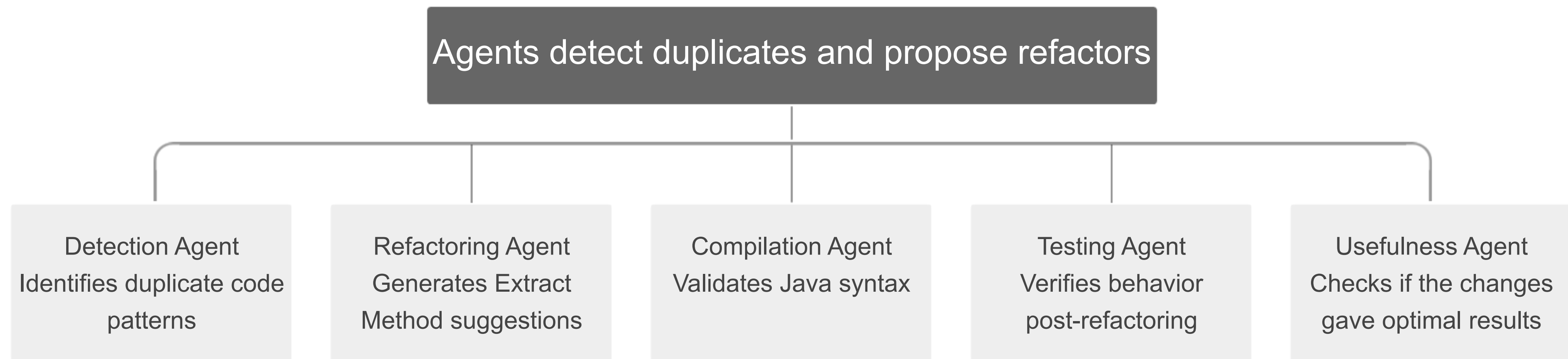
Peer Review

Pull request workflow for code review

Git Commands

clone, pull, add, commit, push, branch, PR

Multi-Agent System Architecture



Detection Agent Workflow

1. On Paste: Checks for clones

Detection agent (3 panelists) analyzes the rest of the code in comparison to what was pasted and try to find clone ranges. If no clones are identified then no further action is taken.

2. Curator

Merges each of the panelists' outputs and returns status, identified clone ranges, and refactoring type.

Panelists

Each one is an independent LLM functioning as a code reviewer. 3 panelists total.

LLMs Used

GPT 5.2, Claude Opus 4.6, Gemini 3 Pro, DeepSeek V3.2; RAG used to augment their performance
IF LLMs fail, PSI*-fallback used

Refactoring Agent Workflow

1. Extract-Method Refactoring

Agent utilizes clone ranges provided by the detection agent as its 3 panelists each propose a refactored piece of code to replace the clone.

2. Curation

The curator analyzes each of the panelists' proposed code and picks the strongest output.

3. Testing

Agent rewrites file replacing refactored code and runs it to test its functionality. Usefulness, compilation, and testing agents are used in conjunction.

4. Final Result

If refactored code fails or is not useful, error is used as feedback and refactoring process is restarted. Once the refactored code is successful it is outputted.

Usefulness Agent Workflow

1. Checks for Changes

The agent will make a check that changes have been made to the code in some capacity

2. Checks for refactoring

The agent will ensure that refactoring has occurred

3. Function Check

The agent will check that the duplicate code has been properly refactored into a function

4. Implementation Check

Finally, the agent will ensure that the refactoring has been properly implemented and that duplicate code does not remain

Compilation and Testing Agents

Compilation Agent

Simply ensures that the newly refactored code still compiles without errors

Testing Agent

Runs multiple test cases to make sure that the refactoring had no effects on the code's functionality

Target Audience & Requirements

Target Users

Java developers and students seeking code quality improvement

Tool Characteristics

Clear, fast, and trustworthy with minimal workflow disruption

Detection Display

Show detected code with file location and line numbers

Change Visualization

Display proposed refactoring changes with compilation and testing validation status

User Actions

Apply refactoring, Edit suggestions, View Diff comparison, or Reject changes



The tool empowers developers to improve code quality efficiently while maintaining full control

Prototype Evaluation and Selection

Prototype Creation

Team created multiple design prototypes for side-by-side comparison

Clarity Assessment

Evaluated usability and visual clarity across all design options

Detail Communication

Assessed how well each design communicates and displays information

Final Selection

Chose design that best displays clone type, location, diff preview, and validation

Prototype: IntelliJ Inline Refactoring Assistant

- Follows the professor's structure: header, explanation, diff view, metadata, action bar

- Designed to feel familiar inside IntelliJ

- Adds AI Trust/Explainability to increase developer confidence

- Keeps the developer in control before applying the refactoring

The screenshot shows the IntelliJ IDEA interface with the `UserValidator.java` file open. The `createUser` method contains duplicate validation logic. The `[AntiCopyPaster] Refactoring Assistant` is displayed, suggesting the extraction of this logic into a new method named `validateInput(...)`.

1 Header

2 Explanation

AntiCopyPaster detected that the code you pasted is a Type-2 clone of the existing method in the same class.

- Existing clone location: `UserValidator.java:42-48`
- Pasted clone location: `UserValidator.java:101-107`
- Suggested method name: `validateInput(...)`

The assistant suggests extracting the common validation logic into a new method and replacing the duplicate with a method call.

3 Diff View (Preview)

Original Duplicate Code

```
- if (u.getUsername() == null || u.getUsername().trim().isEmpty()) {  
-   u.getUsername().trim().isEmpty()) {  
-   throw new IllegalArgumentException("Username is required");  
- }  
- if (u.getEmail() == null || !EMAIL_PATTERN.matcher(u.getEmail()).matches()) {  
-   throw new IllegalArgumentException("Invalid email");  
- }  
- if (u.getPassword() == null || u.getPassword().length() < 8) {  
-   throw new IllegalArgumentException("Password must be at least 8 chars");  
- }
```

Extracted (Replacement)

```
+ validateInput(u, EMAIL_PATTERN);  
+ ...
```

4 Details & Provenance (Confidence: 94%)

Clone Type: Type-2 Useful Extract Method Compiles Tests Passed

5 Action Bar

Apply Edit... Preview Diff Reject Help

Multi-Agent Workflow Behind the Assistant

Detection Agent (Finds duplicate code clones) → Refactoring Agent (Proposes Extract Method refactoring) → Usefulness Checker (Validates usefulness of the refactoring) → Compilation Agent (Compiles the project with the change) → Testing Agent (Runs tests to ensure behavior is preserved)

AI Trust / Explainability

94% Confidence High Safety Score Behavior Preserved Semantic Equivalence 1 Duplicate Removed Compilation Issues

Rationale: The duplicated validation logic is structurally identical except for formatting/whitespace.

Roadmap and Next Steps

Design Refinement

Polish and finalize the selected prototype design

GitHub Organization

Start utilizing it and working on branches

UI Implementation

Advance front-end development of the tool interface

Progress Tracking

Establish a website to monitor and communicate project progress